ATIVIDADE AVALIATIVA

UNIDADE I

Desenvolvimento e Teste de um Mini Sistema de Biblioteca

Testes Unitários • Testes de Integração • Persistência • Cobertura

Disciplina	Testes de Software I
Curso	Ciência da Computação
Aluno(a)	
Matrícula	
Data de entrega	

Objetivo central

Projetar, implementar, executar e interpretar uma estratégia progressiva de testes, iniciando por unidades isoladas e avançando até a integração entre componentes reais do sistema.

1. Objetivo da atividade

Nesta atividade avaliativa, o aluno deverá desenvolver um **Mini Sistema de Biblioteca** e construir progressivamente sua **suíte de testes**. O desenvolvimento deve seguir os conceitos, exemplos, estudos dirigidos e atividades práticas trabalhados durante a Unidade I.

O objetivo não é apenas fazer o sistema funcionar. A atividade deve demonstrar que você (aluno) compreendeu como selecionar casos de teste, estruturar testes automatizados, isolar dependências e verificar a integração entre componentes.

- identificar unidades de código que precisam ser testadas;
- criar testes unitários;
- aplicar o padrão Arrange–Act–Assert;
- utilizar pytest e unittest;
- aplicar parametrização, classes de equivalência e valores-limite;
- testar exceções e comportamentos inválidos;
- utilizar fixtures;
- utilizar mocks e patch quando uma dependência deve ser isolada;
- distinguir teste unitário de teste de integração;
- testar a comunicação entre componentes reais;
- realizar testes envolvendo persistência;
- interpretar falhas e verificar cobertura de testes.

2. Sistema que deverá ser desenvolvido

Você deverá desenvolver um Mini Sistema de Biblioteca capaz, no mínimo, de manter livros e controlar sua disponibilidade para empréstimos e devoluções.

Dados mínimos de um livro

- ISBN
- título
- autor
- quantidade de exemplares disponíveis

Operações mínimas

- cadastrar um livro;
- consultar um livro pelo ISBN;
- verificar a disponibilidade de um livro;
- realizar o empréstimo de um livro;
- realizar a devolução de um livro;
- impedir empréstimos quando não houver exemplares disponíveis;
- impedir operações inválidas quando o livro não existir;
- validar os dados necessários para o cadastro;
- armazenar e recuperar os dados dos livros.

Uma possível organização do projeto é:

```
biblioteca/
│
├── livro.py
├── biblioteca.py
├── repository.py
├── validators.py
└── database.py

tests/
│
├── test_livro.py
├── test_biblioteca.py
├── test_validadores.py
├── test_repository.py
└── test_integracao.py
```

A organização não precisa ser exatamente esta. Entretanto, deve existir separação clara entre o código da aplicação e o código dos testes.

3. Exemplo de referência utilizado nas questões

Nas questões seguintes, o exemplo resolvido utiliza um mini sistema simples. Sua tarefa será transferir o mesmo princípio para o Mini Sistema de Biblioteca.

```
def add2(x):
    return x + 2


def divisao(x, y):
    return x / y


def informacoes():
    return {
        "nome": "Carlos",
        "idade": 20
    }
```

Questão 1 – Construção e organização inicial do sistema

Exemplo de referência – Mini Sistema

Antes de testar add2(), divisao() ou informacoes(), precisamos possuir um código organizado e identificar claramente aquilo que será testado.

```
# mini_sistema.py

def add2(x):
    return x + 2


def divisao(x, y):
    return x / y


def informacoes():
    return {"nome": "Carlos", "idade": 20}
```

```
projeto/
│
├── mini_sistema.py
└── test_mini_sistema.py
```

Sua tarefa

Desenvolva a primeira versão do Mini Sistema de Biblioteca. Implemente, no mínimo, as funcionalidades abaixo.

- classe ou estrutura Livro;
- classe ou estrutura Biblioteca;
- cadastro de livro;
- consulta de livro;
- verificação de disponibilidade;
- empréstimo;
- devolução;
- validação das regras básicas do domínio.

> **Entregue: (a) a estrutura de diretórios; (b) o código desenvolvido; (c) uma breve explicação da responsabilidade de cada módulo, classe ou função criada.**

Questão 2 – Primeiros testes unitários com pytest

Exemplo de referência – Mini Sistema

Um teste unitário verifica uma unidade de comportamento de forma controlada. O padrão Arrange–Act–Assert ajuda a organizar a preparação, a execução e a verificação do resultado.

```
from mini_sistema import add2


def test_add2():
    # Arrange
    valor = 3

    # Act
    resultado = add2(valor)

    # Assert
    assert resultado == 5
```

```
from mini_sistema import informacoes


def test_informacoes():
    resultado = informacoes()

    assert isinstance(resultado, dict)
    assert "nome" in resultado
    assert "idade" in resultado
    assert resultado["idade"] == 20
```

Sua tarefa

Crie testes unitários utilizando pytest para testar isoladamente as principais operações do Mini Sistema de Biblioteca.

- criação de um livro;
- ISBN;
- título;
- autor;
- quantidade de exemplares;
- cadastro de livro;
- consulta de livro existente;
- disponibilidade de livro;
- empréstimo;
- devolução.

> **Em pelo menos dois testes, identifique explicitamente quais linhas correspondem ao Arrange, Act e Assert.**

Questão 3 – Testes utilizando unittest

Exemplo de referência – Mini Sistema

O mesmo comportamento também pode ser testado utilizando unittest. O objetivo do teste não muda; muda a estrutura fornecida pelo framework.

```
import unittest
from mini_sistema import add2


class TestAdd2(unittest.TestCase):

    def test_add2(self):
        resultado = add2(3)
        self.assertEqual(resultado, 5)


if __name__ == "__main__":
    unittest.main()
```

Sua tarefa

Escolha pelo menos três funcionalidades do Mini Sistema de Biblioteca que já foram testadas com pytest e implemente novamente seus testes utilizando unittest. Utilize adequadamente recursos como self.assertEqual(), self.assertTrue(), self.assertFalse() e self.assertIsInstance().

Ao final, explique brevemente a principal diferença que você observou entre a escrita de testes utilizando pytest e unittest.

Questão 4 – Parametrização, classes de equivalência e valores-limite

Exemplo de referência – Mini Sistema

Não devemos verificar add2() utilizando somente uma entrada. A parametrização permite executar o mesmo raciocínio de teste com vários conjuntos de dados.

```
import pytest
from mini_sistema import add2


@pytest.mark.parametrize(
    "entrada, esperado",
    [
        (0, 2),
        (1, 3),
        (5, 7),
        (-1, 1),
        (-10, -8),
    ]
)
def test_add2_parametrizado(entrada, esperado):
    assert add2(entrada) == esperado
```

Sua tarefa

Escolha pelo menos duas regras do sistema de biblioteca e desenvolva testes parametrizados. Uma delas deverá obrigatoriamente estar relacionada à quantidade de exemplares.

Considere valores como -1, 0, 1 e 2 e identifique valores válidos, inválidos, valores-limite e classes de equivalência.

Entrada	Classe	Resultado esperado
...	válida / inválida / limite	...
...	...	...

Construa os testes utilizando @pytest.mark.parametrize(...) e explique o critério usado para selecionar os dados de teste.

Questão 5 – Testando exceções

Exemplo de referência – Mini Sistema

Um teste pode ser aprovado justamente porque a execução gerou a exceção esperada.

```
def divisao(x, y):
    return x / y
```

```
import pytest
from mini_sistema import divisao


def test_divisao_por_zero():
    with pytest.raises(ZeroDivisionError):
        divisao(10, 0)
```

```
with self.assertRaises(ZeroDivisionError):
    divisao(10, 0)
```

Sua tarefa

Implemente regras no sistema que produzam exceções em situações inválidas e teste os cenários a seguir.

- cadastro com dados inválidos;
- cadastro duplicado de livro;
- empréstimo de livro inexistente;
- empréstimo quando a quantidade disponível for zero;
- devolução de livro inexistente.

Utilize pytest.raises() e escolha pelo menos um cenário para implementar também com assertRaises(). Explique por que receber a exceção esperada representa sucesso do teste.

Questão 6 – Utilização de fixtures

Exemplo de referência – Mini Sistema

Fixtures permitem reutilizar a preparação de dados ou objetos necessários por vários testes.

```
import pytest


@pytest.fixture
def dados_usuario():
    return {
        "nome": "Carlos",
        "idade": 20
    }
```

```
def test_nome(dados_usuario):
    assert dados_usuario["nome"] == "Carlos"


def test_idade(dados_usuario):
    assert dados_usuario["idade"] == 20


def test_tipo(dados_usuario):
    assert isinstance(dados_usuario, dict)
```

Sua tarefa

Crie pelo menos duas fixtures para o sistema de biblioteca e reutilize cada uma em diferentes testes.

- possíveis fixtures: livro, livro_disponivel, livro_indisponivel, biblioteca ou repositorio;
- mostre o código de cada fixture;
- utilize cada fixture em pelo menos dois testes;
- explique qual repetição de código foi eliminada.

Questão 7 – Isolamento de dependências com Mock/Patch

Exemplo de referência – Mini Sistema

Quando o objetivo é testar somente uma unidade, uma dependência pode ser substituída temporariamente para que o teste permaneça isolado.

```
def montar_relatorio():
    dados = informacoes()
    return f"{dados['nome']} - {dados['idade']} anos
```

```
from unittest.mock import patch


@patch("mini_sistema.informacoes")
def test_montar_relatorio(mock_informacoes):
    mock_informacoes.return_value = {
        "nome": "Ana",
        "idade": 25
    }

    resultado = montar_relatorio()

    assert resultado == "Ana - 25 anos
```

Sua tarefa

Identifique uma dependência existente no Mini Sistema de Biblioteca e utilize Mock ou patch para isolá-la. A dependência pode ser um repositório, uma consulta, a persistência ou outra classe.

- configure pelo menos um return_value;
- verifique que determinado método foi chamado;
- utilize, quando adequado, assert_called_once() ou assert_called_once_with(...);
- explique por que o teste continua sendo unitário mesmo quando o código original possui dependências.

Questão 8 – Comportamentos diferentes da dependência

Exemplo de referência – Mini Sistema

Mocks também podem simular retornos sucessivos ou falhas, permitindo testar a reação da unidade a cenários distintos.

```
mock_funcao.side_effect = [
    10,
    20,
    30
]
```

```
mock_funcao.side_effect = Exception("Falha simulada")
```

Sua tarefa

Utilize side_effect ou mecanismo equivalente para testar pelo menos dois comportamentos diferentes de uma dependência do Mini Sistema de Biblioteca. Um deverá representar uma situação normal e outro uma situação de falha.

- indique qual dependência foi simulada;
- explique por que foi utilizada uma simulação;
- informe o resultado esperado para cada cenário.

PARTE II – TESTES DE INTEGRAÇÃO

A partir desta etapa, o objetivo muda: nos testes anteriores procuramos isolar unidades; agora queremos verificar se componentes reais conseguem trabalhar corretamente em conjunto.

```
TESTE UNITÁRIO
componente isolado
    ↓
dependências controladas / simuladas

TESTE DE INTEGRAÇÃO
componente real → componente real → resultado observado
```

Questão 9 – Primeiro teste de integração

Exemplo de referência – Mini Sistema

Quando funções ou componentes reais são utilizados juntos, sem substituir as dependências por mocks, o teste passa a verificar a integração entre essas partes.

```
informacoes()
    ↓
montar_relatorio()
    ↓
resultado final
```

```
def test_integracao_relatorio():
   resultado = montar_relatorio()

   assert "Carlos" in resultado
   assert "20" in resultado
```

Sua tarefa

Crie um teste de integração para o fluxo abaixo.

- cadastrar um livro;
- pesquisar o livro cadastrado;
- confirmar que existem exemplares disponíveis;
- realizar o empréstimo;
- consultar novamente o livro;
- confirmar que a quantidade disponível foi reduzida.

> **Não utilize mocks nesse teste. Todos os componentes envolvidos no fluxo deverão ser reais.**

Questão 10 – Integração do fluxo completo

Exemplo de referência – Mini Sistema

Um teste de integração pode acompanhar um fluxo maior, verificando se a saída de um componente é corretamente utilizada pelo seguinte.

```
informacoes()
    ↓
processamento
    ↓
montar_relatorio()
    ↓
salvar_relatorio()
```

Sua tarefa

Implemente e teste o fluxo completo: cadastrar livro → consultar livro → realizar empréstimo → consultar novamente → realizar devolução → consultar novamente.

- verifique que o livro é cadastrado corretamente;

- verifique que o livro pode ser encontrado;
- confirme que o número de exemplares diminui após o empréstimo;
- confirme que o número de exemplares aumenta após a devolução;
- verifique que o estado final corresponde ao esperado;
- explique quais componentes estão sendo integrados.

Questão 11 – Teste de integração com persistência

Exemplo de referência – Mini Sistema

Em um teste de integração com persistência, não basta verificar se um método salvar() foi chamado. É necessário verificar se os dados foram efetivamente armazenados e recuperados.

```
gerar informação
    ↓
processar
    ↓
salvar
    ↓
ler novamente
```

Sua tarefa

Implemente uma forma simples de persistência utilizando SQLite e um repositório responsável por armazenar livros.

- inicialize o banco de testes;
- cadastre um livro;
- persista o livro;
- realize uma nova consulta ao banco;
- recupere o livro;
- compare os dados recuperados com os dados cadastrados.

```
# Sugestão para o banco de testes
conexao = sqlite3.connect(":memory:")
```

> **O banco utilizado pelos testes não deverá interferir em um eventual banco da aplicação.**

Questão 12 – Fixture para testes de integração

Exemplo de referência – Mini Sistema

Fixtures podem preparar e desmontar recursos utilizados durante os testes de integração.

```
@pytest.fixture
def ambiente():
    # setup
    recurso = criar_recurso()

    yield recurso

    # teardown
    encerrar_recurso(recurso)
```

Sua tarefa

Crie uma fixture responsável por preparar o ambiente dos testes de integração. Ela poderá criar o banco, criar tabelas, criar o repositório ou cadastrar dados iniciais.

- utilize a fixture em pelo menos dois testes de integração;
- explique a função do setup;
- explique a execução do teste;
- explique a função do teardown.

Questão 13 – Investigação de falhas

Exemplo de referência – Mini Sistema

Uma boa suíte de testes deve detectar um defeito introduzido no código, e não apenas produzir mensagens PASSED.

```
def add2(x):
    return x + 2
```

```
# Erro proposital
def add2(x):
    return x + 3
```

```
def test_add2():
    assert add2(2) == 4

# Resultado esperado do framework:
# assert 5 == 4
```

Sua tarefa

Escolha uma funcionalidade do Mini Sistema de Biblioteca e introduza propositalmente um erro. Execute novamente os testes e documente a investigação.

- erro introduzido;
- teste que detectou o problema;
- mensagem apresentada pelo framework;
- interpretação da falha;
- correção realizada.

> **Exemplos de mutações: não diminuir o estoque no empréstimo, aumentar dois exemplares na devolução, permitir empréstimo com estoque zero ou consultar um ISBN incorreto.**

Questão 14 – Cobertura de testes

Exemplo de referência – Mini Sistema

A cobertura ajuda a identificar partes do código que ainda não foram exercitadas pelos testes. Entretanto, uma cobertura elevada não garante, isoladamente, que os testes são bons.

```
python -m pytest --cov=mini_sistema
```

Sua tarefa

Execute uma ferramenta de cobertura sobre o Mini Sistema de Biblioteca. Utilize o nome correspondente ao pacote criado por você.

- apresente o percentual de cobertura obtido;
- identifique módulos com menor cobertura;
- identifique linhas ou comportamentos importantes ainda não testados;
- crie pelo menos um novo teste após analisar o relatório;
- execute novamente a cobertura e compare os resultados;
- responda por que uma cobertura elevada, isoladamente, não garante a qualidade dos testes.

4. Execução dos testes

A suíte deverá ser executada pelo terminal. Exemplos:

```
python -m pytest -v

python -m pytest -v -s

python -m pytest --cov=biblioteca
```

Os testes desenvolvidos com unittest também deverão ser executáveis.

5. Entrega da atividade

O projeto deverá conter, no mínimo:

```
Código-fonte do Mini Sistema de Biblioteca
      +
Testes unitários
      +
Testes parametrizados
      +
Testes de exceções
      +
Fixtures
      +
Testes utilizando Mock/Patch
      +
Testes de integração
      +
Testes de persistência
      +
Relatório de cobertura
      +
Respostas e explicações solicitadas
```

Além do código, entregue um documento com a sequência de desenvolvimento. Para cada questão, apresente:

1. objetivo do teste;
2. código desenvolvido;
3. resultado da execução;
4. interpretação do resultado.

> **Não é suficiente apresentar somente o código. A avaliação considera a capacidade de explicar o que está sendo testado, por que o teste foi construído daquela maneira e como interpretar seu resultado.**

6. Critérios de avaliação

Critério	Pontuação
Organização e funcionamento do Mini Sistema de Biblioteca	1,0
Testes unitários com pytest e unittest	1,5
Parametrização, equivalência e valores-limite	1,0
Testes de exceções	0,5
Fixtures	0,5
Mock/Patch e isolamento de dependências	1,0
Testes de integração	2,0
Integração com persistência	1,0
Cobertura e investigação de falhas	0,5
Organização, explicações e interpretação dos resultados	1,0
TOTAL	**10,0**

7. Questão final de síntese

Após concluir toda a atividade, responda:

> **Explique, utilizando exemplos do Mini Sistema de Biblioteca desenvolvido por você, a diferença entre um teste unitário e um teste de integração.**

Na resposta, indique:

- qual componente foi testado isoladamente;
- quais dependências foram substituídas ou controladas;
- quais componentes foram utilizados de forma real no teste de integração;
- qual tipo de defeito cada categoria de teste poderia identificar.

Síntese da progressão esperada

```
Desenvolvimento do sistema
        ↓
Teste unitário simples
        ↓
pytest / unittest
        ↓
Parametrização
        ↓
Equivalência e valores-limite
        ↓
Exceções
        ↓
Fixtures
        ↓
Mock / Patch
        ↓
Teste de integração
        ↓
Fluxo completo
        ↓
Persistência
        ↓
Investigação de falhas
        ↓
Cobertura
        ↓
Análise dos resultados
```

Resultado esperado da atividade: demonstrar que o aluno é capaz de projetar, implementar, executar e interpretar uma estratégia progressiva de testes de software, e não apenas produzir código que “passa” nos testes.